PulsePass — Product Requirements Document

Plataforma de eventos, artistas y entradas
Caso de estudio académico — Fase 3: Capa de Controladores REST

Documento	**PRD - PulsePass — Capa de Controladores REST**
Versión	1.0
Estado	Aprobado para taller académico
Tecnologías objetivo	Java 21 / Spring Boot 4 / Spring MVC / Bean Validation / JUnit 5 / Mockito / MockMvc
Alcance técnico	API REST, validación HTTP, manejo de errores y pruebas de Controller

Documento de requisitos del producto

Control del documento

Campo	**Detalle**
Propósito	Definir los requisitos funcionales, contratos HTTP, criterios de aceptación y estrategia de pruebas para exponer mediante una API REST los casos de uso de PulsePass implementados previamente en las capas de Persistencia y Servicios.
Audiencia	Estudiantes, docentes y revisores técnicos del taller de capa de controladores.
Dependencias	Este documento continúa el PRD de Persistencia y el PRD de Capa de Servicios. No redefine el modelo de dominio ni las reglas de negocio ya aprobadas; las consume y expone mediante HTTP.
Alcance	MVP académico enfocado en Controllers REST, validación de requests, códigos HTTP, manejo uniforme de errores y pruebas aisladas con MockMvc.
Fuera de alcance	Autenticación, autorización, Spring Security, JWT, OAuth2, frontend, pasarela de pagos, mensajería, notificaciones, OpenAPI/Swagger, pruebas E2E y despliegue productivo.

1. Resumen ejecutivo

PulsePass es una plataforma para descubrir eventos y gestionar entradas para conciertos, festivales, conferencias, eventos universitarios, competencias deportivas y actividades culturales.

La implementación del caso de estudio se desarrolla de forma incremental:

```
Fase 1 — Persistencia
    ↓
Modelo relacional
Entidades JPA
Repositories
Flyway
```

```
PostgreSQL
Testcontainers

Fase 2 — Servicios
    ↓
Reglas de negocio
DTOs
MapStruct
Transacciones
Excepciones
Unit tests

Fase 3 — Controladores REST
    ↓
Endpoints HTTP
Validación de entrada
Códigos HTTP
Manejo de errores
MockMvc
```

Este PRD corresponde exclusivamente a la **Fase 3**.

La capa Controller deberá transformar solicitudes HTTP en llamadas a los Services ya definidos y transformar las respuestas o excepciones de la aplicación en respuestas HTTP consistentes.

> **Principio de producto**
> La API REST debe exponer el comportamiento de PulsePass sin duplicar reglas de negocio, sin acceder directamente a repositories y sin exponer entidades JPA.

2. Visión y problema

2.1 Problema

Las capas de Persistencia y Servicios permiten almacenar información y ejecutar reglas de negocio, pero todavía no existe un mecanismo HTTP que permita a clientes externos:

- consultar venues;
- consultar y crear eventos;
- publicar eventos;
- asociar artistas;
- registrar usuarios;
- comprar tickets;
- consultar tickets;
- cancelar tickets;
- marcar tickets como usados.

Además, los errores de negocio deben transformarse en respuestas HTTP predecibles y los requests inválidos deben rechazarse antes de llegar a la lógica de negocio.

2.2 Visión

Construir una API REST clara y consistente sobre los Services existentes, de manera que PulsePass pueda ser consumido posteriormente por:

```
Frontend web
Aplicación móvil
Cliente administrativo
Integraciones externas
```

sin acoplar estos clientes al modelo de persistencia.

2.3 Objetivos del MVP de Controller

- Exponer los casos de uso existentes mediante endpoints REST.
- Utilizar correctamente `GET`, `POST` y `PATCH`.
- Utilizar códigos HTTP semánticamente correctos.
- Validar requests mediante Bean Validation.
- Retornar exclusivamente DTOs.
- Centralizar el manejo de errores.
- Transformar excepciones de Service en respuestas HTTP.
- Probar el contrato HTTP de forma aislada.
- Garantizar cobertura de todos los métodos definidos en los Services del PRD anterior.

2.4 No objetivos

- Implementar nuevas reglas de negocio.
- Modificar el modelo relacional.
- Implementar nuevos repositories.
- Acceder a PostgreSQL desde Controller.
- Implementar autenticación o autorización.
- Procesar pagos reales.
- Generar QR.
- Implementar frontend.
- Crear documentación OpenAPI.
- Ejecutar pruebas E2E.
- Usar Testcontainers en las pruebas de Controller.

3. Relación con los PRD anteriores

Este documento debe considerarse una extensión directa de los dos PRD anteriores.

3.1 Fase 1 — Persistencia

La primera fase definió:

```
Venue
Event
Artist
User
```

```
UserProfile
Ticket
```

y sus relaciones:

```
Venue 1 ──────── N Event
Event N ──────── M Artist
User 1 ──────── 1 UserProfile
User 1 ──────── N Ticket
Event 1 ──────── N Ticket
```

También definió constraints, consultas y persistencia mediante PostgreSQL, Flyway y Spring Data JPA.

3.2 Fase 2 — Servicios

La segunda fase agregó:

```
Service interfaces
Service implementations
DTO request/response
MapStruct
Reglas de negocio
Transacciones
Excepciones personalizadas
Unit tests
```

Los Controllers deberán consumir estos Services sin duplicar comportamiento.

3.3 Fase 3 — Controladores

Esta fase agrega exclusivamente la frontera HTTP:

```
HTTP Request
    ↓
Controller
    ↓
Service
    ↓
Persistencia
```

y:

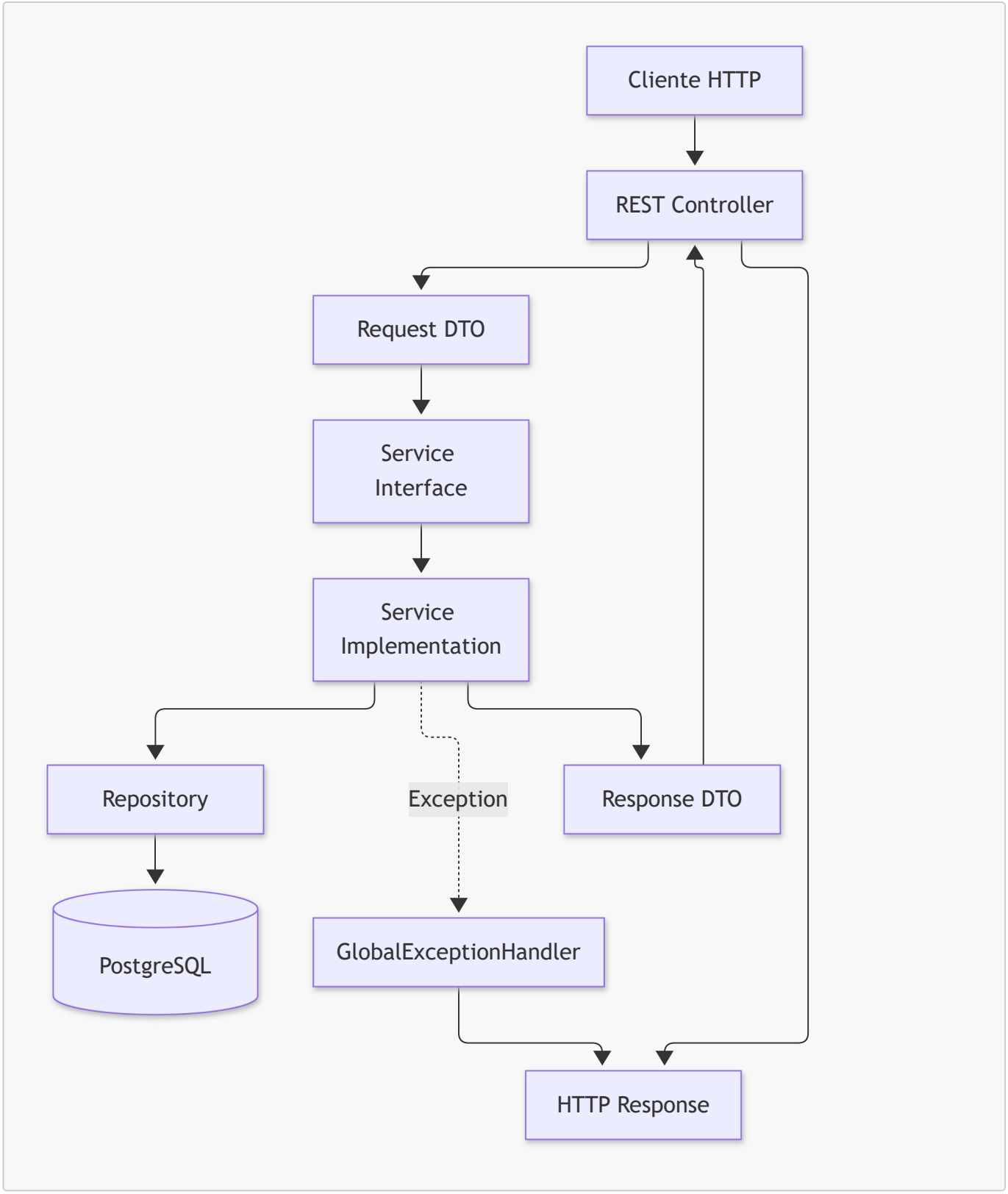
```
Service Response / Exception
    ↓
Controller / Exception Handler
```

```
    ↓
HTTP Response
```

4. Actores y necesidades

Actor	Necesidad principal
Asistente / usuario	Consultar eventos, registrarse, comprar y consultar tickets mediante HTTP.
Operación de eventos	Consultar venues, crear eventos, publicarlos y asociar artistas.
Soporte / analista	Consultar usuarios, tickets y ventas pagadas de un evento.
Cliente frontend	Consumir una API consistente basada en JSON y códigos HTTP predecibles.
Equipo de desarrollo	Mantener separación entre Controller, Service y Repository.
Equipo QA	Validar contratos HTTP mediante tests automatizados y reproducibles.

5. Arquitectura esperada



La separación obligatoria es:

```
Controller
    ↓
HTTP concerns

Service
    ↓
Business rules
```

```
Repository
    ↓
Persistence
```

6. Principios de diseño

ID	Principio
CTRL-001	El Controller no implementa reglas de negocio.
CTRL-002	El Controller no accede directamente a Repository.
CTRL-003	Los endpoints no exponen entidades JPA.
CTRL-004	Los contratos públicos utilizan DTOs.
CTRL-005	La validación estructural pertenece a Bean Validation.
CTRL-006	Las reglas de negocio permanecen en Service.
CTRL-007	El manejo de excepciones debe estar centralizado.
CTRL-008	Los errores deben usar un contrato JSON uniforme.
CTRL-009	Los Controllers deben ser delgados y delegar al Service.
CTRL-010	Cada endpoint debe poseer pruebas de contrato HTTP.

7. Alcance funcional del MVP

El MVP de Controller se divide en cinco capacidades.

7.1 Gestión de venues

ID	Requisito	Descripción	Prioridad	Criterio de aceptación
FR-CTRL-VEN-001	Consultar venue por código	Exponer un endpoint que consulte un venue mediante su código de negocio.	Must	Un venue existente retorna `200 OK`; uno inexistente retorna `404 Not Found`.
FR-CTRL-VEN-002	Consultar venues activos	Exponer los venues activos.	Must	Retorna `200 OK` con una colección de `VenueResponse`.

7.2 Gestión de eventos

ID	Requisito	Descripción	Prioridad	Criterio de aceptación
FR-CTRL-EVT-001	Crear evento	Recibir `CreateEventRequest` y delegar en `EventService.create`.	Must	Request válido retorna `201 Created`.
FR-CTRL-	Consultar evento	Consultar por `eventCode`.	Must	Recurso existente retorna `200`; inexistente retorna `404`.

ID	Requisito	Descripción	Prioridad	Criterio de aceptación
EVT-002				
FR-CTRL-EVT-003	Consultar publicados	Listar eventos publicados.	Must	Retorna `200 OK` con `EventSummaryResponse`.
FR-CTRL-EVT-004	Publicar evento	Exponer la transición de publicación ya implementada en Service.	Must	Operación válida retorna `200`; conflicto de negocio retorna `409`.
FR-CTRL-EVT-005	Asociar artista	Asociar un artista existente a un evento.	Must	Asociación válida retorna `200`; conflicto retorna `409`.
FR-CTRL-EVT-006	Buscar por artista	Consultar eventos por `stageName`.	Must	Retorna `200 OK` con los eventos encontrados.

7.3 Gestión de artistas

ID	Requisito	Descripción	Prioridad	Criterio de aceptación
FR-CTRL-ART-001	Consultar artista por ID	Exponer `ArtistService.findById`.	Must	Existente retorna `200`; inexistente `404`.
FR-CTRL-ART-002	Consultar por stageName	Consultar artista mediante nombre artístico.	Must	Retorna `200` o `404`.
FR-CTRL-ART-003	Consultar activos	Listar artistas activos.	Must	Retorna `200 OK`.

7.4 Gestión de usuarios

ID	Requisito	Descripción	Prioridad	Criterio de aceptación
FR-CTRL-USR-001	Registrar usuario	Recibir `RegisterUserRequest`.	Must	Registro válido retorna `201 Created`.
FR-CTRL-USR-002	Consultar por email	Consultar usuario por email.	Must	Retorna `200` o `404`.
FR-CTRL-USR-003	Consultar por username	Consultar usuario por username.	Must	Retorna `200` o `404`.

7.5 Gestión de tickets

ID	Requisito	Descripción	Prioridad	Criterio de aceptación
FR-CTRL-TKT-001	Comprar ticket	Recibir `PurchaseTicketRequest`.	Must	Compra válida retorna `201 Created`.

ID	Requisito	Descripción	Prioridad	Criterio de aceptación
FR-CTRL-TKT-002	Consultar ticket	Consultar ticket por código.	Must	Retorna `200` o `404`.
FR-CTRL-TKT-003	Tickets por usuario	Consultar tickets mediante email.	Must	Retorna `200 OK`.
FR-CTRL-TKT-004	Tickets pagados por evento	Consultar tickets `PAID` de un evento.	Must	Retorna `200 OK`.
FR-CTRL-TKT-005	Cancelar ticket	Exponer `TicketService.cancel`.	Must	Transición válida retorna `200`; inválida `409`.
FR-CTRL-TKT-006	Usar ticket	Exponer `TicketService.markAsUsed`.	Must	Transición válida retorna `200`; inválida `409`.

8. Contrato HTTP del MVP

8.1 VenueController

Método	Endpoint	Service	Respuesta exitosa
GET	`/api/venues/{code}`	`findByCode(code)`	`200 OK`
GET	`/api/venues/active`	`findActiveVenues()`	`200 OK`

8.2 EventController

Método	Endpoint	Service	Respuesta exitosa
POST	`/api/events`	`create(request)`	`201 Created`
GET	`/api/events/{eventCode}`	`findByCode(eventCode)`	`200 OK`
GET	`/api/events/published`	`findPublishedEvents()`	`200 OK`
PATCH	`/api/events/{eventCode}/publish`	`publish(eventCode)`	`200 OK`
POST	`/api/events/{eventCode}/artists/{artistId}`	`addArtist(eventCode, artistId)`	`200 OK`
GET	`/api/events/by-artist?stageName=...`	`findByArtist(stageName)`	`200 OK`

8.3 ArtistController

Método	Endpoint	Service	Respuesta exitosa
GET	`/api/artists/{id}`	`findById(id)`	`200 OK`

Método	Endpoint	Service	Respuesta exitosa
GET	`/api/artists/by-stage-name?stageName=...`	`findByStageName(stageName)`	`200 OK`
GET	`/api/artists/active`	`findActiveArtists()`	`200 OK`

8.4 UserController

Método	Endpoint	Service	Respuesta exitosa
POST	`/api/users`	`register(request)`	`201 Created`
GET	`/api/users/by-email?email=...`	`findByEmail(email)`	`200 OK`
GET	`/api/users/by-username?username=...`	`findByUsername(username)`	`200 OK`

8.5 TicketController / EventController

Método	Endpoint	Service	Respuesta exitosa
POST	`/api/tickets`	`purchase(request)`	`201 Created`
GET	`/api/tickets/{ticketCode}`	`findByCode(ticketCode)`	`200 OK`
GET	`/api/tickets/by-user?email=...`	`findByUserEmail(email)`	`200 OK`
GET	`/api/events/{eventCode}/tickets/paid`	`findPaidTicketsByEvent(eventCode)`	`200 OK`
PATCH	`/api/tickets/{ticketCode}/cancel`	`cancel(ticketCode)`	`200 OK`
PATCH	`/api/tickets/{ticketCode}/use`	`markAsUsed(ticketCode)`	`200 OK`

9. Cobertura obligatoria Service → HTTP

La Fase 3 debe cubrir todos los métodos públicos definidos en los Services de la Fase 2.

```
VenueService      2 métodos
EventService      6 métodos
ArtistService     3 métodos
UserService       3 métodos
TicketService     6 métodos
──────────────────────────
TOTAL            20 operaciones
```

No se considera completa la capa Controller si existe un método de Service definido en el PRD anterior sin una forma HTTP de invocarlo.

10. Contratos DTO heredados

La capa Controller reutilizará los DTOs definidos en la Fase 2.

10.1 Request DTOs

```
CreateEventRequest
RegisterUserRequest
PurchaseTicketRequest
```

10.2 Response DTOs

```
VenueResponse
EventResponse
EventSummaryResponse
ArtistResponse
UserResponse
TicketResponse
```

Los Controllers no deben retornar:

```
Venue
Event
Artist
User
UserProfile
Ticket
```

11. Validación de entrada

La validación estructural debe ejecutarse antes de invocar el Service.

11.1 CreateEventRequest

Debe validar como mínimo:

Campo	Validación
eventCode	`@NotBlank`
name	`@NotBlank`
description	longitud máxima definida
category	`@NotNull`
eventDate	`@NotNull`

Campo	Validación
minimumAge	`@NotNull`, `@Min(0)`
venueCode	`@NotBlank`

11.2 RegisterUserRequest

Debe validar como mínimo:

Campo	Validación
username	`@NotBlank`
email	`@NotBlank`, `@Email`
firstName	`@NotBlank`
lastName	`@NotBlank`
birthDate	`@NotNull`

11.3 PurchaseTicketRequest

Debe validar como mínimo:

Campo	Validación
userEmail	`@NotBlank`, `@Email`
eventCode	`@NotBlank`
type	`@NotNull`

11.4 Límite entre Controller y Service

Controller valida:

```
campo obligatorio
formato email
longitud
valor mínimo
JSON mal formado
```

Service valida:

```
usuario activo
evento existente
evento publicado
edad mínima
capacidad
transiciones de estado
duplicados de negocio
```

12. Reglas de negocio heredadas

Los Controllers **no reimplementan** las reglas definidas en la capa Service.

Entre las reglas heredadas se encuentran:

- un usuario inactivo no puede comprar;
- solo se compra para eventos `PUBLISHED`;
- no se compra para eventos pasados;
- el usuario debe cumplir la edad mínima;
- no puede superarse la capacidad del venue;
- el último ticket disponible puede llevar el evento a `SOLD_OUT`;
- solo un ticket `PAID` puede cancelarse;
- un ticket `USED` no puede cancelarse;
- un ticket `CANCELLED` no puede marcarse como usado;
- un artista no puede asociarse dos veces al mismo evento;
- no deben agregarse artistas a eventos `CANCELLED` o `FINISHED`.

El Controller únicamente debe traducir los resultados de estas reglas al contrato HTTP.

13. Contrato estándar de errores

Todos los errores HTTP producidos por la aplicación deberán utilizar:

```
public record ErrorResponse(
        LocalDateTime timestamp,
        int status,
        String error,
        String message,
        Map<String, String> details
) {
}
```

Ejemplo:

```
{
  "timestamp": "2026-10-04T20:00:00",
  "status": 404,
```

```
  "error": "Not Found",
  "message": "Event not found: CMF-2026",
  "details": {}
}
```

Para errores de validación:

```
{
  "timestamp": "2026-10-04T20:00:00",
  "status": 400,
  "error": "Bad Request",
  "message": "Validation failed",
  "details": {
    "eventCode": "Event code is required",
    "venueCode": "Venue code is required"
  }
}
```

14. Mapeo de excepciones a HTTP

Excepción / condición	Código HTTP	Uso
`MethodArgumentNotValidException`	400	Bean Validation
Request mal formado	400	JSON inválido
`ResourceNotFoundException`	404	Recurso inexistente
`DuplicateResourceException`	409	Conflicto de unicidad
`BusinessRuleException`	409	Regla de negocio incumplida
Excepción inesperada	500	Error interno no controlado

Todos los handlers deberán retornar:

```
ResponseEntity<ErrorResponse>
```

15. Manejo global de excepciones

La aplicación debe implementar un único componente:

```
GlobalExceptionHandler
```

con:

```
@RestControllerAdvice
```

Los Controllers no deben repetir bloques:

```
try {
   ...
} catch (...) {
   ...
}
```

para convertir errores de dominio a HTTP.

16. Códigos HTTP requeridos

Código	Uso
`200 OK`	Consultas y actualizaciones correctas
`201 Created`	Creación de evento, usuario o ticket
`400 Bad Request`	Request inválido o mal formado
`404 Not Found`	Recurso inexistente
`409 Conflict`	Duplicado o regla de negocio incumplida
`500 Internal Server Error`	Error inesperado

17. Restricciones técnicas del caso académico

- Java 21.
- Spring Boot 4.x.
- Spring MVC.
- Maven.
- DTOs con Java `record`.
- Bean Validation.
- `@RestController`.
- `@RequestMapping`.
- `@GetMapping`.
- `@PostMapping`.
- `@PatchMapping`.
- `@PathVariable`.
- `@RequestParam`.
- `@RequestBody`.
- `@Valid`.
- `ResponseEntity`.
- `@RestControllerAdvice`.

- `@ExceptionHandler`.
- JUnit 5.
- Mockito.
- MockMvc.
- `@WebMvcTest`.
- `@MockitoBean`.
- `jsonPath`.

No utilizar:

```
Repository dentro de Controller
Entity como response HTTP
reglas de negocio dentro de Controller
Testcontainers para unit tests de Controller
@SpringBootTest cuando @WebMvcTest sea suficiente
```

18. Estructura esperada

```
src/main/java/com/pulsepass
│
├── controller
│   ├── VenueController.java
│   ├── EventController.java
│   ├── ArtistController.java
│   ├── UserController.java
│   └── TicketController.java
│
├── dto
│   ├── request
│   │   ├── CreateEventRequest.java
│   │   ├── RegisterUserRequest.java
│   │   └── PurchaseTicketRequest.java
│   │
│   └── response
│       ├── VenueResponse.java
│       ├── EventResponse.java
│       ├── EventSummaryResponse.java
│       ├── ArtistResponse.java
│       ├── UserResponse.java
│       ├── TicketResponse.java
│       └── ErrorResponse.java
│
├── exception
│   ├── ResourceNotFoundException.java
│   ├── DuplicateResourceException.java
│   ├── BusinessRuleException.java
│   └── GlobalExceptionHandler.java
│
└── service
```

Tests:

```
src/test/java/com/pulsepass/controller
│
├── VenueControllerTest.java
├── EventControllerTest.java
├── ArtistControllerTest.java
├── UserControllerTest.java
└── TicketControllerTest.java
```

19. Casos de uso HTTP clave

ID	Caso de uso	Endpoint principal	Resultado esperado
UC-CTRL-01	Consultar venue	`GET /api/venues/{code}`	Venue en JSON
UC-CTRL-02	Crear evento	`POST /api/events`	Evento creado
UC-CTRL-03	Publicar evento	`PATCH /api/events/{code}/publish`	Evento publicado
UC-CTRL-04	Asignar artista	`POST /api/events/{code}/artists/{id}`	Evento actualizado
UC-CTRL-05	Consultar cartelera	`GET /api/events/published`	Eventos publicados
UC-CTRL-06	Registrar usuario	`POST /api/users`	Usuario creado
UC-CTRL-07	Comprar ticket	`POST /api/tickets`	Ticket `PAID`
UC-CTRL-08	Consultar tickets	`GET /api/tickets/by-user`	Tickets del usuario
UC-CTRL-09	Cancelar ticket	`PATCH /api/tickets/{code}/cancel`	Ticket cancelado
UC-CTRL-10	Validar ingreso	`PATCH /api/tickets/{code}/use`	Ticket usado

20. Criterios de aceptación funcionales

ID	Escenario de aceptación
AC-CTRL-001	Dado `VEN-SMR-01`, cuando se consulta `GET /api/venues/VEN-SMR-01`, entonces la API retorna `200 OK` y un `VenueResponse`.
AC-CTRL-002	Dado un código de venue inexistente, cuando se consulta, entonces la API retorna `404` usando `ErrorResponse`.
AC-CTRL-003	Dado un `CreateEventRequest` válido, cuando se envía a `POST /api/events`, entonces retorna `201 Created`.
AC-CTRL-004	Dado un request de evento inválido, cuando Bean Validation falla, entonces retorna `400` y `details` contiene los campos inválidos.
AC-CTRL-005	Dado `CMF-2026` en `DRAFT`, cuando el Service permite publicarlo, entonces `PATCH /publish` retorna `200`.

ID	Escenario de aceptación
AC-CTRL-006	Cuando Service lanza `BusinessRuleException`, la API retorna `409 Conflict`.
AC-CTRL-007	Dado un `RegisterUserRequest` válido, `POST /api/users` retorna `201`.
AC-CTRL-008	Dado email duplicado, la API retorna `409` sin exponer detalles internos.
AC-CTRL-009	Dada una compra válida, `POST /api/tickets` retorna `201` y `TicketResponse`.
AC-CTRL-010	Dado un usuario menor de edad, la capa Service rechaza la compra y Controller retorna `409`.
AC-CTRL-011	Dado un ticket `PAID`, cancelar retorna `200`.
AC-CTRL-012	Dado un ticket `USED`, cancelar retorna `409`.
AC-CTRL-013	Dado un ticket `PAID`, marcarlo como usado retorna `200`.
AC-CTRL-014	Dado un ticket `CANCELLED`, marcarlo como usado retorna `409`.
AC-CTRL-015	Ningún Controller accede directamente a un Repository.
AC-CTRL-016	Ningún endpoint retorna entidades JPA.

21. Estrategia de pruebas y calidad

Las pruebas de Controller deben validar el comportamiento HTTP sin iniciar PostgreSQL ni ejecutar la lógica real de Service.

La unidad bajo prueba es:

```
HTTP Request
      ↓
Controller
      ↓
Mock Service
      ↓
HTTP Response
```

Debe utilizarse:

```
@WebMvcTest
MockMvc
@MockitoBean
when(...)
verify(...)
verify(..., never())
jsonPath(...)
```

21.1 Criterios de calidad

ID	Criterio
QT-CTRL-001	Cada Controller posee una clase de pruebas.
QT-CTRL-002	Los Services son mocks.
QT-CTRL-003	Se valida status HTTP.
QT-CTRL-004	Se valida `Content-Type`.
QT-CTRL-005	Se validan campos JSON relevantes con `jsonPath`.
QT-CTRL-006	Se verifica la interacción con Service.
QT-CTRL-007	Request inválido no invoca Service.
QT-CTRL-008	Se prueban respuestas `404`.
QT-CTRL-009	Se prueban respuestas `409`.
QT-CTRL-010	Se prueba Bean Validation con `400`.
QT-CTRL-011	Se prueba `ErrorResponse`.
QT-CTRL-012	`mvn clean test` finaliza con `BUILD SUCCESS`.

22. Tests mínimos requeridos

22.1 VenueController

```
TEST-CTRL-VEN-001  venue existente → 200
TEST-CTRL-VEN-002  venue inexistente → 404
TEST-CTRL-VEN-003  venues activos → 200
```

22.2 EventController

```
TEST-CTRL-EVT-001  crear válido → 201
TEST-CTRL-EVT-002  request inválido → 400
TEST-CTRL-EVT-003  consultar existente → 200
TEST-CTRL-EVT-004  inexistente → 404
TEST-CTRL-EVT-005  listar publicados → 200
```

```
TEST-CTRL-EVT-006  publicar válido → 200
TEST-CTRL-EVT-007  publicar inválido → 409
TEST-CTRL-EVT-008  asociar artista → 200
TEST-CTRL-EVT-009  buscar por artista → 200
```

22.3 ArtistController

```
TEST-CTRL-ART-001  buscar por ID → 200
TEST-CTRL-ART-002  ID inexistente → 404
TEST-CTRL-ART-003  buscar por stageName → 200
TEST-CTRL-ART-004  listar activos → 200
```

22.4 UserController

```
TEST-CTRL-USR-001  registrar válido → 201
TEST-CTRL-USR-002  email inválido → 400
TEST-CTRL-USR-003  username duplicado → 409
TEST-CTRL-USR-004  consultar por email → 200
TEST-CTRL-USR-005  consultar por username → 200
```

22.5 TicketController

```
TEST-CTRL-TKT-001  compra válida → 201
TEST-CTRL-TKT-002  request inválido → 400
TEST-CTRL-TKT-003  usuario inexistente → 404
TEST-CTRL-TKT-004  regla de negocio → 409
TEST-CTRL-TKT-005  consultar ticket → 200
TEST-CTRL-TKT-006  tickets por usuario → 200
TEST-CTRL-TKT-007  tickets PAID por evento → 200
TEST-CTRL-TKT-008  cancelar válido → 200
TEST-CTRL-TKT-009  cancelar inválido → 409
TEST-CTRL-TKT-010  usar válido → 200
TEST-CTRL-TKT-011  usar inválido → 409
```

23. Datos de referencia del escenario

Se mantiene el mismo escenario usado en los PRD anteriores.

23.1 Venue

Campo	Valor
code	VEN-SMR-01
name	Marina Convention Center

Campo	Valor
city	Santa Marta
capacity	3
active	true

23.2 Evento principal

Campo	Valor
eventCode	`CMF-2026`
name	Caribbean Music Fest 2026
category	MUSIC
status inicial	DRAFT
minimumAge	18

23.3 Artistas

```
Solar Beat
Neon Waves
Caribbean Sound
```

23.4 Usuarios

```
Andrea — adulta — active=true
Carlos — adulto — active=true
Laura — 17 años — active=true
Miguel — adulto — active=false
```

Este escenario debe reutilizarse para mantener continuidad entre Persistencia, Service y Controller.

24. Requisitos no funcionales

ID	Requisito
NFR-CTRL-001	Separación de responsabilidades: Controller no contiene lógica de negocio.
NFR-CTRL-002	Testabilidad: Controller puede probarse sin PostgreSQL ni ApplicationContext completo.
NFR-CTRL-003	Consistencia: todos los errores usan `ErrorResponse`.
NFR-CTRL-004	Mantenibilidad: cada Controller depende únicamente de Services necesarios.
NFR-CTRL-005	Legibilidad: nombres de endpoints y métodos reflejan el lenguaje del dominio.

ID	Requisito
NFR-CTRL-006	Inmutabilidad: DTOs continúan implementándose preferiblemente como `record`.
NFR-CTRL-007	Seguridad de datos: no exponer stack traces ni detalles internos al cliente.
NFR-CTRL-008	Determinismo de pruebas: los tests de Controller no dependen de infraestructura externa.

25. Anti-patrones prohibidos

25.1 Repository dentro del Controller

No permitido:

```
private final EventRepository eventRepository;
```

25.2 Reglas de negocio

No permitido:

```
if (event.getStatus() != EventStatus.PUBLISHED) {
    ...
}
```

25.3 Entidades como response

No permitido:

```
ResponseEntity<Event>
```

25.4 Manejo repetido de excepciones

No se deben implementar múltiples bloques `try/catch` en cada endpoint.

25.5 Código 200 para todo

No deben representarse como `200`:

```
creación
validación fallida
recurso inexistente
conflicto de negocio
error interno
```

25.6 Acceso a Entity desde JSON

El cliente no debe enviar objetos JPA completos para expresar asociaciones.

Ejemplo incorrecto:

```
{
  "venue": {
    "id": 12
  }
}
```

Se deben utilizar identificadores de negocio definidos por los DTOs:

```
{
  "venueCode": "VEN-SMR-01"
}
```

26. Riesgos y supuestos

ID	Riesgo / supuesto
R-CTRL-001	La API depende de que la Fase 2 esté implementada de acuerdo con su PRD.
R-CTRL-002	Cambios futuros en firmas de Service pueden exigir cambios en los contratos HTTP.
R-CTRL-003	El MVP no implementa autenticación; todos los endpoints son conceptualmente públicos.
R-CTRL-004	No se incluye paginación; listas grandes deberán resolverse en una evolución futura.
R-CTRL-005	No se versiona la API (`/api/v1`) en este MVP académico.
R-CTRL-006	Las reglas de negocio no deben inferirse desde códigos HTTP; permanecen documentadas en Service.

27. Fuera de alcance del taller

- Spring Security.
- JWT.
- OAuth2.
- Roles y permisos.
- Rate limiting.
- OpenAPI/Swagger.
- Paginación.
- Sorting dinámico.

- HATEOAS.
- CORS avanzado.
- API Gateway.
- Frontend.
- Kafka.
- RabbitMQ.
- Email/SMS.
- Pagos.
- QR.
- Pruebas E2E.
- Testcontainers en Controller.
- Deployment cloud.

28. Evolución futura sugerida

La API podrá evolucionar posteriormente hacia:

- `/api/v1`.
- Spring Security + JWT.
- roles `ADMIN`, `ORGANIZER`, `USER`.
- OpenAPI.
- paginación.
- filtros dinámicos.
- idempotencia de compras.
- rate limiting.
- observabilidad.
- correlation IDs.
- auditoría.
- API Gateway.
- integración con frontend React.
- pruebas de integración completas.
- pruebas contractuales.
- pruebas E2E.

29. Matriz de trazabilidad resumida

Requisito Controller	Service de origen	Regla / comportamiento	Test principal
FR-CTRL-VEN-001	`VenueService.findByCode`	Recurso existente/inexistente	TEST-CTRL-VEN-001/002
FR-CTRL-VEN-002	`VenueService.findActiveVenues`	Solo activos	TEST-CTRL-VEN-003
FR-CTRL-EVT-001	`EventService.create`	Reglas de creación heredadas	TEST-CTRL-EVT-001/002

Requisito Controller	Service de origen	Regla / comportamiento	Test principal
FR-CTRL-EVT-002	`EventService.findByCode`	ResourceNotFound	TEST-CTRL-EVT-003/004
FR-CTRL-EVT-003	`EventService.findPublishedEvents`	Solo publicados	TEST-CTRL-EVT-005
FR-CTRL-EVT-004	`EventService.publish`	Transición de estado	TEST-CTRL-EVT-006/007
FR-CTRL-EVT-005	`EventService.addArtist`	Asociación no duplicada	TEST-CTRL-EVT-008
FR-CTRL-EVT-006	`EventService.findByArtist`	Búsqueda por artista	TEST-CTRL-EVT-009
FR-CTRL-ART-*	`ArtistService`	Consulta de artistas	TEST-CTRL-ART-*
FR-CTRL-USR-001	`UserService.register`	Validaciones + duplicados	TEST-CTRL-USR-001/002/003
FR-CTRL-USR-002/003	`UserService`	Consulta usuario	TEST-CTRL-USR-004/005
FR-CTRL-TKT-001	`TicketService.purchase`	Compra, edad, capacidad, estado	TEST-CTRL-TKT-001..004
FR-CTRL-TKT-002	`TicketService.findByCode`	Consulta	TEST-CTRL-TKT-005
FR-CTRL-TKT-003	`TicketService.findByUserEmail`	Consulta por usuario	TEST-CTRL-TKT-006
FR-CTRL-TKT-004	`TicketService.findPaidTicketsByEvent`	Solo PAID	TEST-CTRL-TKT-007
FR-CTRL-TKT-005	`TicketService.cancel`	PAID → CANCELLED	TEST-CTRL-TKT-008/009
FR-CTRL-TKT-006	`TicketService.markAsUsed`	PAID → USED	TEST-CTRL-TKT-010/011

30. Definition of Done

La capa de Controladores se considera terminada cuando:

```
[ ] Existen VenueController, EventController, ArtistController, UserController y
TicketController

[ ] Los cinco Controllers dependen exclusivamente de Services
```

```
[ ] Todos los Controller responden un Objeto ResponseEntity

[ ] Ningún Controller depende directamente de Repository

[ ] Ningún Controller implementa reglas de negocio

[ ] Ningún endpoint retorna entidades JPA

[ ] Se reutilizan DTOs definidos en la capa Service

[ ] Se cubren los 20 métodos públicos de Service

[ ] POST de creación retorna 201

[ ] GET exitoso retorna 200

[ ] PATCH exitoso retorna 200

[ ] Bean Validation retorna 400

[ ] ResourceNotFoundException retorna 404

[ ] DuplicateResourceException retorna 409

[ ] BusinessRuleException retorna 409

[ ] Error inesperado retorna 500

[ ] Todos los errores utilizan ErrorResponse

[ ] Todos los handlers retornan ResponseEntity<ErrorResponse>

[ ] Existe GlobalExceptionHandler con @RestControllerAdvice

[ ] Las pruebas utilizan @WebMvcTest

[ ] Los Services se reemplazan con @MockitoBean

[ ] Las pruebas utilizan MockMvc

[ ] Las pruebas validan status, content type y JSON

[ ] Se utiliza verify(...) para validar llamadas a Service

[ ] Se utiliza verify(..., never()) cuando un request inválido no debe llegar a
Service

[ ] mvn clean test finaliza con BUILD SUCCESS

[ ] El README documenta los endpoints principales y cómo ejecutar las pruebas
```

31. Preguntas para el equipo

1. ¿Qué responsabilidad pertenece al Controller y cuál al Service?
2. ¿Por qué un Controller no debería acceder directamente a Repository?
3. ¿Qué diferencia existe entre una validación estructural y una regla de negocio?
4. ¿Cuándo debe utilizarse `400`, `404` y `409`?
5. ¿Por qué una creación debería retornar `201` en lugar de `200`?
6. ¿Por qué los errores deben utilizar una estructura uniforme?
7. ¿Qué ventaja aporta `@RestControllerAdvice` frente a capturar excepciones en cada endpoint?
8. ¿Por qué `@WebMvcTest` es más apropiado que `@SpringBootTest` para estas pruebas?
9. ¿Cómo verificarías que un request inválido nunca invoca el Service?
10. ¿Qué cambios serían necesarios para agregar autenticación JWT en una fase posterior?

32. Resumen final

PulsePass se considera correctamente especificado para la **Fase 3 — Capa de Controladores REST** cuando los estudiantes pueden tomar los casos de uso y reglas ya implementados en la capa Service y exponerlos mediante contratos HTTP claros, consistentes y verificables.

Este PRD no redefine el dominio ni la lógica de negocio. Su propósito es completar la evolución incremental iniciada en Persistencia y continuada en Service:

```
Persistencia
    ↓
Datos consistentes

Service
    ↓
Reglas de negocio

Controller
    ↓
API REST
```

El documento define el **QUÉ** debe ofrecer la API, los contratos HTTP mínimos, los códigos de respuesta, la estrategia de validación y los criterios de aceptación. El estudiante debe implementar el **CÓMO** respetando la separación de responsabilidades establecida en las fases anteriores.